Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Oleh:

Muhammad Nafis Habibi	13524018
Kloce Paul William Saragih	13524040

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Daftar Isi **2**
Penjelasan Singkat Program **3**
Algoritma **3**
1. Voxelization 3
2. Renderering 6
Pengujian **9**
Kesimpulan dan Saran **15**
Lampiran **17**

Penjelasan Singkat Program

Program ini melakukan proses *voxelization* pada sebuah objek 3 dimensi dalam file .obj, yaitu mengubah objek tersebut menjadi objek yang terdiri dari kubus-kubus kecil. Proses *voxelization* dilakukan dengan strategi algoritma *Divide and Conquer*.

Program dibuat dalam bahasa C++ dan dipercepat dengan melakukan *multithreading*. Terdapat juga program *renderer* untuk melihat objek 3d dari file .obj.

Algoritma

1. *Voxelization*

   Proses *voxelization* dilakukan dengan strategi algoritma *Divide and Conquer*, yaitu dengan memecah kubus yang lebih besar menjadi 8 kubus yang lebih kecil. Sebuah kubus diletakkan di suatu ruang jika dan hanya jika terdapat perpotongan dengan objek yang dimasukkan. Jika kubus memiliki perpotongan dengan salah satu permukaan pada objek, kubus tersebut akan dipecah menjadi lebih kecil. Jika tidak ada perpotongan, kubus dihilangkan dan tidak perlu dipecah lagi. Setiap kubus kecil yang terbentuk akan dicek lagi apakah berpotongan dengan objek tersebut. Hal ini dilakukan secara rekursif hingga kedalaman maksimal yang ditentukan. Pada kedalaman maksimal, titik-titik sudut pada kubus serta segitiga yang membentuk permukaan kubus disimpan sebagai output.

   Di awal program, cari satu kubus yang berisikan seluruh titik dari objek. Kubus dibuat pada koordinat terkecil sampai koordinat terbesar dari objek. Jika dari koordinat terkecil hingga terbesar hanya terbentuk balok, perpanjang balok hingga membentuk kubus. Kubus ini akan menjadi kasus pertama dalam rekursi.

   Terdapat tiga hal yang perlu diperiksa untuk menentukan apakah sebuah segitiga berpotongan dengan atau berada di dalam kubus:

   a. Apakah salah satu titik sudut segitiga berada di dalam kubus?
   b. Apakah salah satu sisi pada segitiga berpotongan dengan salah satu sisi pada kubus?
   c. Apakah salah satu rusuk pada kubus berpotongan dengan daerah segitiga?

   Poin pertama (a) dapat diperiksa dengan menentukan apakah masing-masing koordinat x, y, dan z dari titik sudut segitiga berada di antara koordinat minimum dan maksimum dari kubus. Jika iya, maka kubus berpotongan dengan segitiga.

Poin kedua (b) dapat diperiksa dengan mencari perpotongan garis dari sisi segitiga yang diperpanjang dengan bidang yang diperluas dari sisi pada kubus. Lalu, periksa apakah perpotongan berada di segmen sisi segitiga tersebut dan berada di daerah sisi pada kubus. Jika iya, maka kubus berpotongan dengan segitiga.

Poin ketiga (c) kurang lebih dapat diselesaikan dengan cara yang sama dengan poin kedua. Cari terlebih dahulu perpotongan garis dari rusuk kubus yang diperpanjang dengan bidang 3 dimensi yang diperluas dari segitiga. Lalu, periksa apakah perpotongan berada di segmen rusuk kubus tersebut dan berada di daerah segitiga. Jika iya, maka kubus berpotongan dengan segitiga.

Untuk mencari perpotongan bidang dengan garis, perlu dicari rumus dari bidang dan garis tersebut. Jika diketahui tiga titik p1, p2, dan p3 yang terletak pada bidang, rumus bidang dapat dicari dengan cara berikut:

a. Hitung perkalian *cross* dari (p2 - p1) dan (p3 - p1). Hasilnya adalah vektor normal dari bidang yang diinginkan. Misalkan hasilnya adalah v_n = (a, b, c)
b. Hitung nilai d dari persamaan bidang ax + by + cz + d = 0. Nilai a, b, dan c adalah nilai dari vektor normal. Substitusikan salah satu titik pada persamaan di atas sebagai nilai x, y, dan z untuk mendapatkan nilai d. Misalkan memakai p1, maka d = -(a * p1.x + b * p1.y + c * p1.z).

Jika diketahui dua titik p1 dan p2 yang terletak pada bidang, rumus garis adalah persamaan parametrik dari p1 + t * (p2 - p1). Misalkan v = p2 - p1, maka:

- x = p1.x + t * v.x
- y = p1.y + t * v.x
- z = p1.z + t * v.x

Untuk mendapatkan titik potong bidang dengan garis, subtitusikan x, y, dan z pada persamaan bidang, maka t = - (a * p1.x + b * p1.y + c * p1.z + d) / (a * v.x + b * v.y + c * v.z)

Terakhir, nilai t yang didapat disubtitusikan ke dalam persamaan parametrik garis untuk menghasilkan titik perpotongan antara bidang dengan garis. Perlu hati-hati jika garis sejajar dengan bidang. Bisa diperiksa dengan perkalian *dot* antara vektor normal bidang (a, b, c) dengan v. Jika hasil kali mendekati nol, garis dapat dianggap sejajar dengan bidang dan tidak ada titik potong antara keduanya.

Misalkan pt adalah titik potong yang didapat. Untuk menentukan apakah hasil perpotongan berada di segmen garis yang sebenarnya (di antara p1 dan p2), hitung perkalian *dot* dari (p2 - p1) dengan (pt - p1) dan (p1 - p2) dengan (pt - p2). Jika keduanya positif, dapat disimpulkan bahwa titik potong berada di segmen garis tersebut.

Untuk menentukan apakah hasil perpotongan berada di daerah segitiga (asumsi titik potong berada di bidang yang dibentuk dari perluasan segitiga tersebut), hitung perkalian *cross* dengan tiap sisi segitiga. Jika vektor hasil perkalian mengarah ke arah yang sama, dapat disimpulkan bahwa titik potong berada di daerah segitiga tersebut. Dalam implementasi, vektor dinormalkan terlebih dahulu, lalu dibandingkan dengan vektor lain yang dinormalkan juga. Jika kurang lebih nilai vektor sama, berarti orientasi sama dan dapat disimpulkan bahwa titik potong berada di daerah segitiga tersebut.

Untuk menentukan apakah hasil perpotongan berada di daerah sisi kubus, cukup mengecek dua sumbu saja sesuai dengan orientasi sisi kubus tersebut karena sisi kubus selalu menghadap ke salah satu sumbu. Misalkan sisi kubus menghadap ke arah sumbu z, cukup koordinat x dan y yang dibandingkan dengan koordinat dari sisi kubus. Jika koordinat berada di antara koordinat minimum dan maksimum dari kubus, dapat disimpulkan bahwa titik potong tersebut berada di sisi kubus tersebut.

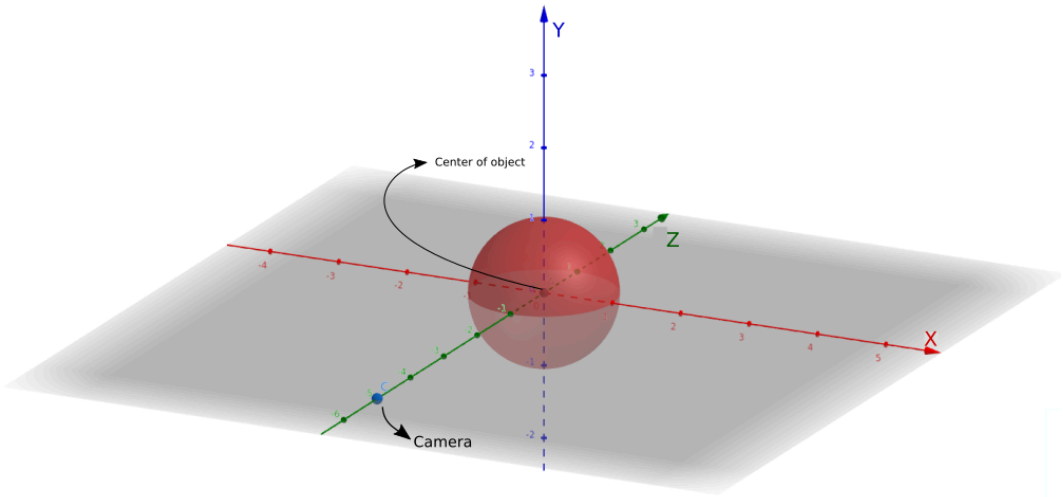
Saat melakukan rekursi, semua segitiga yang diperiksa apakah berpotongan dengan kubus di-*list* kembali. Hanya segitiga yang berpotongan yang akan diperiksa lagi pada kedalaman rekursi selanjutnya untuk kubus yang lebih kecil. Jika besar kubus sudah lebih kecil dari segitiga, segitiga yang perlu diperiksa apakah berpotongan akan semakin sedikit sehingga pada kedalaman yang lebih dalam, perhitungan akan semakin cepat.

Untuk mempercepat perhitungan juga, setiap cabang rekursi akan dibuat *thread* masing-masing untuk menghitungnya. Namun, *thread* yang terlalu banyak dapat menyebabkan program justru semakin lama karena *overhead* dari *context switch* nya atau program error karena gagal membuat *thread* baru. Supaya *thread* tidak terlalu banyak, pembuatan *thread* hanya dilakukan pada kedalaman yang kecil. Dalam implementasi, hanya tiga kali percabangan yang membuat *thread* anak dengan total maksimal *thread* sekitar 8^3 = 512. Dalam kenyataannya, jumlah *thread* dapat lebih kecil karena tidak semua cabang rekursi akan bercabang lagi.

2. *Renderering*

Proses *rendering* diawali dengan membaca input *.obj* dan mengklasifikasikan tiap data sebagai *face* maupun *vertex*. Setelah menerima input *faces* dan *vertices*, program akan melakukan translasi pada data-data tersebut sehingga *center* (titik tengah) dari objek akan berada pada titik (0,0,0) pada koordinat kartesius pada 3D-*space*. Selanjutnya, program akan melakukan normalisasi posisi tiap verteks agar berada pada rentang [-1,1] inklusif. Hal ini dilakukan dengan cara mencari dimensi maksimum pada objek, dan melakukan *rescaling* sehingga *aspect-ratio* dari objek tetap dipertahankan.

Untuk melakukan rendering, sebuah kamera akan diletakkan pada posisi (0,0,-5) pada koordinat kartesius. Program yang dibuat akan menggunakan FOV sebesar 45 derajat. Berikut adalah ilustrasi posisi kamera dan objek:



Untuk implementasi fitur interaktif yakni *zoom* dan *rotation*. Maka diperlukan matriks transformasi untuk melakukan perhitungan posisi tiap verteks. Agar mempermudah proses perhitungan, akan digunakan vektor homogen, sehingga setiap matriks transformasi akan berukuran 4x4. Untuk fitur zoom, diperlukan matriks transformasi linear rotasi sebagai berikut (program menggunakan konvensi ZYX) :

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Rotation Matrix

(For X-Axis Rotation)

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Rotation Matrix

(For Y-Axis Rotation)

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Rotation Matrix

(For Z-Axis Rotation)

Sumber : https://www.gatevidyalay.com/3d-rotation-in-computer-graphics-definition-examples/

Untuk fitur zoom, akan dibuat sebuah matriks dilatasi 4x4 sebagai berikut:

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Scaling Matrix

Sumber : https://www.gatevidyalay.com/3d-scaling-in-computer-graphics-definition-examples/

Untuk melakukan proyeksi posisi tiap *vertex* ke layar, diperlukan sebuah matriks *projection*, sehingga posisi akhir titik pada layar dapat dihitung sebagai berikut:

$$\begin{bmatrix} X_{pos} \\ Y_{pos} \\ Z_{pos} \\ W_{pos} \end{bmatrix} = \mathrm{M}_{projection} \cdot M_{zoom} \cdot M_{rotX} \cdot M_{rotY} \cdot M_{rotZ} \cdot \begin{bmatrix} X_p \\ Y_p \\ Z_p \\ 1 \end{bmatrix}$$

Matriks proyeksi yang digunakan adalah matriks proyeksi perspektif (*perspective projection matrix*) yang memetakan koordinat 3D dari ruang kamera ke koordinat yang kemudian dinormalisasi ke rentang [-1, 1]. Proyeksi perspektif menciptakan efek kedalaman di mana objek yang lebih jauh dari kamera akan terlihat lebih kecil, sesuai dengan cara pandangan manusia.

Matriks proyeksi perspektif 4×4 yang digunakan dalam program adalah:

P[0][0] = 1 / (aspectRatio × tan(fov/2))

P[1][1] = 1 / tan(fov/2)

P[2][2] = - (far + near) / (far - near)

P[2][3] = - (2 × far × near) / (far - near)

P[3][2] = -1

P[3][3] = 0

Di mana:

- **fov** = *field of view* vertikal dalam radian (program menggunakan π/4 atau 45°)
- **aspectRatio** = lebar layar / tinggi layar
- **near** = jarak bidang potong dekat dari kamera (program menggunakan 0.1)
- **far** = jarak bidang potong jauh dari kamera (program menggunakan 100.0)

Elemen-elemen matriks ini memiliki fungsi sebagai berikut:

1. P[0][0] mengatur skala horizontal. Nilai ini memperhitungkan *aspect ratio* layar sehingga objek tidak terdistorsi pada layar dengan proporsi yang berbeda.

2. P[1][1] mengatur skala vertikal berdasarkan *field of view*. Semakin kecil FOV, semakin besar nilai P[1][1], yang berarti objek akan terlihat lebih diperbesar.

3. P[2][2] dan P[2][3] mengatur pemetaan kedalaman (koordinat z) secara non-linear. Pemetaan ini memastikan bahwa objek yang berada di antara bidang *near* dan *far* dipetakan ke rentang [-1, 1], dengan presisi kedalaman yang lebih tinggi untuk objek yang lebih dekat dengan kamera.

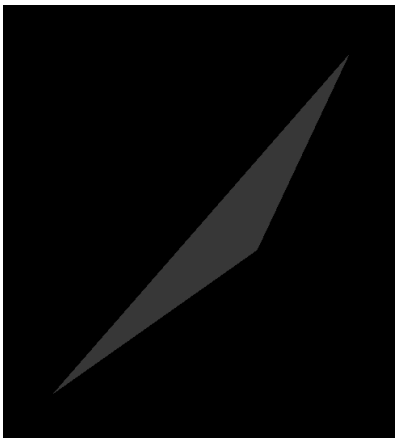
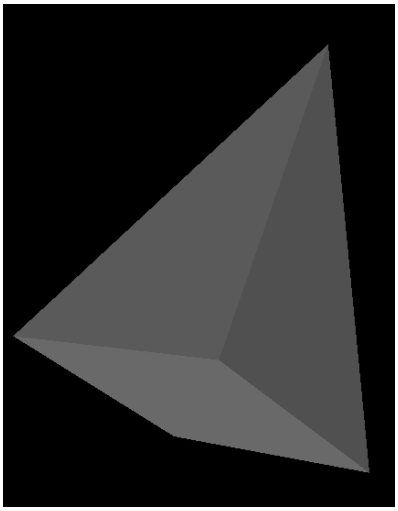
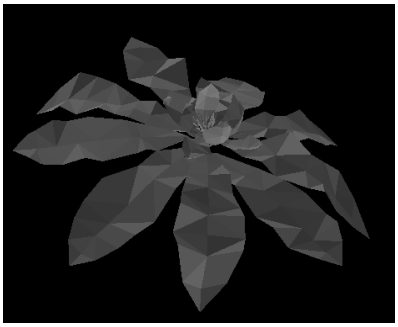
4. **P[3][2] = -1** adalah elemen kunci yang mengaktifkan *perspective divide*. Elemen ini menyalin nilai z (kedalaman) ke komponen w dari koordinat homogen, sehingga setelah pembagian dengan w, objek yang lebih jauh akan memiliki nilai koordinat yang lebih kecil (terlihat lebih kecil di layar).

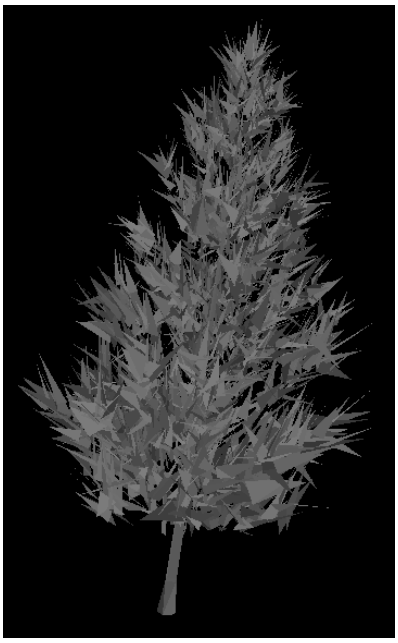
Proses transformasi lengkap dari titik 3D (x, y, z, 1) menggunakan matriks proyeksi menghasilkan koordinat *clip space* (x′, y′, z′, w′) di mana w′ = -z. Kemudian, dilakukan *perspective divide* dengan membagi setiap komponen dengan w′ untuk mendapatkan *Normalized Device Coordinates* (NDC) dalam rentang [-1, 1]. Terakhir, koordinat NDC dipetakan ke koordinat layar dengan transformasi:

- $x_{layar} = (x_{NDC} + 1) \times \text{lebar} / 2$
- $y_{layar} = (1 - y_{NDC}) \times \text{tinggi} / 2$.

Pengujian

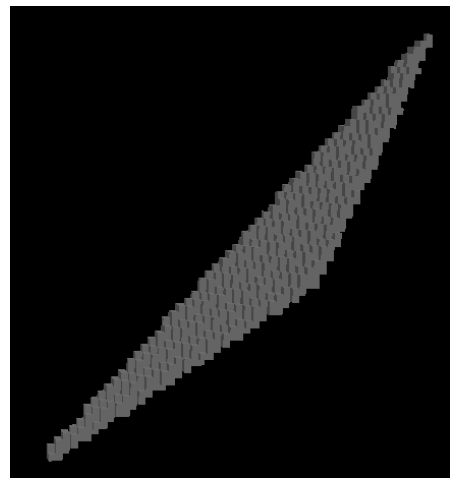
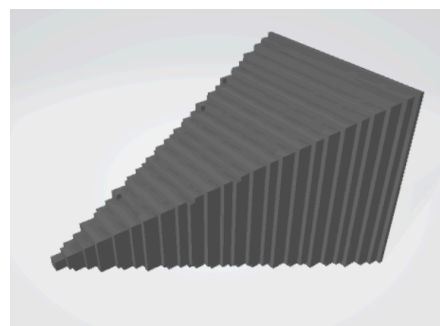
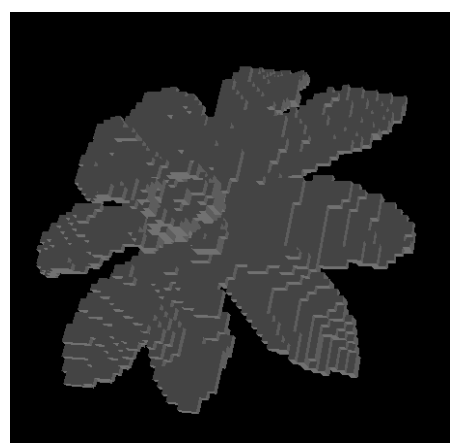
Berikut objek yang akan menjadi kasus uji:

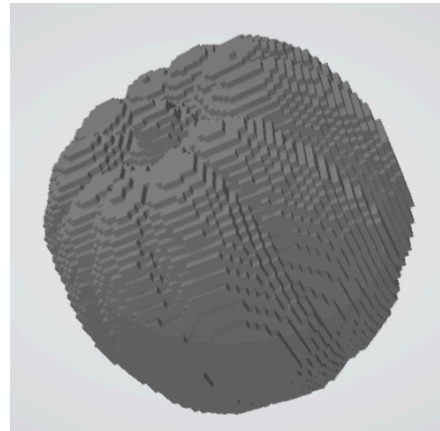
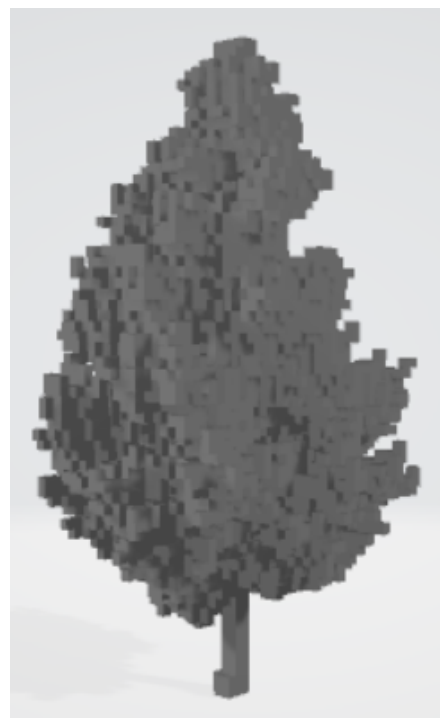
No.	Nama Objek	Jumlah Verteks	Jumlah Face/Triangle	Gambar (dari renderer)
1	triangle.obj	3	1	
2	pyramid.obj	5	6	
3	magnolia.obj	806	1247	

4	pumpkin.obj	5002	10000	
5	pine.obj	7570	5296	

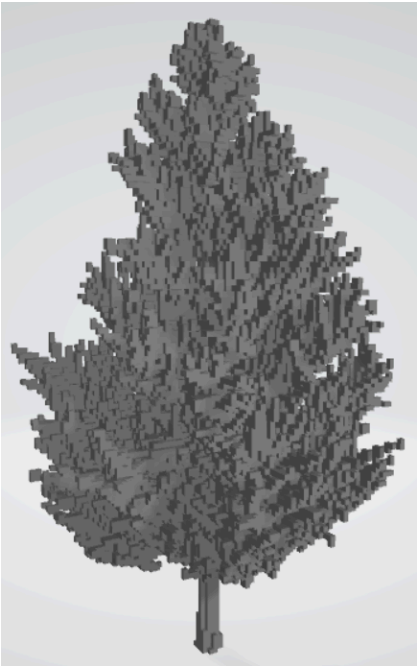
Pengujian tiap objek dengan kedalaman maksimal 6

No.	Nama Objek	Output Terminal	Gambar Hasil

1	triangle.obj	Input the .obj path: test/input/triangle.obj Input max depth: 6 Voxelizing... Finished in 0.033s Outputting to test/output/voxelized-triangle.obj Finished! STATISTICS ========== Max Depth: 6 Voxel Count: 1018 Vertex Count: 8144 Face (Triangle) Count: 12216 Traversed Node in Recursion Tree: Depth 0: 1 Depth 1: 8 Depth 2: 24 Depth 3: 56 Depth 4: 216 Depth 5: 672 Depth 6: 2304 Killed Node in Recursion Tree: Depth 0: 0 Depth 1: 5 Depth 2: 17 Depth 3: 29 Depth 4: 132 Depth 5: 384 Depth 6: 1286	
2	pyramid.obj	Input the .obj path: test/input/pyramid.obj Input max depth: 6 Voxelizing... Finished in 0.228s Outputting to test/output/voxelized-pyramid.obj Finished! STATISTICS ========== Max Depth: 6 Voxel Count: 8944 Vertex Count: 71552 Face (Triangle) Count: 107328 Traversed Node in Recursion Tree: Depth 0: 1 Depth 1: 8 Depth 2: 40 Depth 3: 208 Depth 4: 960 Depth 5: 4136 Depth 6: 17424 Killed Node in Recursion Tree: Depth 0: 0 Depth 1: 3 Depth 2: 14 Depth 3: 88 Depth 4: 443 Depth 5: 1958 Depth 6: 8480	
3	magnolia.obj	Input the .obj path: test/input/magnolia.obj Input max depth: 6 Voxelizing... Finished in 0.447s Outputting to test/output/voxelized-magnolia.obj Finished! STATISTICS ========== Max Depth: 6 Voxel Count: 4398 Vertex Count: 35184 Face (Triangle) Count: 52776 Traversed Node in Recursion Tree: Depth 0: 1 Depth 1: 8 Depth 2: 32 Depth 3: 160 Depth 4: 648 Depth 5: 2144 Depth 6: 8728 Killed Node in Recursion Tree: Depth 0: 0 Depth 1: 4 Depth 2: 12 Depth 3: 79 Depth 4: 380 Depth 5: 1053 Depth 6: 4330	

4	pumpkin.obj	Input the .obj path: test/input/pumpkin.obj Input max depth: 6 Voxelizing... Finished in 2.02s Outputting to test/output/voxelized-pumpkin.obj Finished! STATISTICS ========== Max Depth: 6 Voxel Count: 18608 Vertex Count: 148864 Face (Triangle) Count: 223296 Traversed Node in Recursion Tree: Depth 0: 1 Depth 1: 8 Depth 2: 64 Depth 3: 448 Depth 4: 2024 Depth 5: 8864 Depth 6: 36824 Killed Node in Recursion Tree: Depth 0: 0 Depth 1: 0 Depth 2: 8 Depth 3: 195 Depth 4: 916 Depth 5: 4261 Depth 6: 18216	
5	pine.obj	Input the .obj path: test/input/pine.obj Input max depth: 6 Voxelizing... Finished in 1.496s Outputting to test/output/voxelized-pine.obj Finished! STATISTICS ========== Max Depth: 6 Voxel Count: 12491 Vertex Count: 99928 Face (Triangle) Count: 149892 Traversed Node in Recursion Tree: Depth 0: 1 Depth 1: 8 Depth 2: 40 Depth 3: 176 Depth 4: 832 Depth 5: 4200 Depth 6: 21800 Killed Node in Recursion Tree: Depth 0: 0 Depth 1: 3 Depth 2: 18 Depth 3: 72 Depth 4: 307 Depth 5: 1475 Depth 6: 9309	

Tambahan karena hasilnya bagus:



Hasil dari konversi pine.obj dengan kedalaman maksimal 7 sudah mulai terlihat percabangan daunnya.

Gambar yang memiliki latar belakang hitam adalah gambar dari *renderer* yang kami buat. Namun, *renderer* tidak cukup kuat untuk me-*render* objek dengan jumlah *face* hingga ratusan ribu. Oleh karena itu, sisa gambar diambil dengan aplikasi 3D viewer lain (yang berlatar belakang abu-abu).

Pengujian tiap objek dengan kedalaman maksimal dari 4 sampai 8:

No.	Nama Objek	Kedalaman Maksimal	Waktu Eksekusi (s)	Jumlah Voxel	Jumlah Verteks	Jumlah Face/Triangle
1	triangle.obj	4	0.018	84	672	1008
		5	0.019	288	2304	3456
		6	0.033	1018	8144	12216
		7	0.129	3822	30576	45864
		8	0.286	14787	118296	177444
2	pyramid.obj	4	0.076	517	4136	6204
		5	0.068	2178	17424	26136

		6	0.228	8944	71552	107328
		7	0.495	3822	30576	45864
		8	1.536	143715	1149720	1724580
3	magnolia.obj	4	0.190	268	2144	3216
		5	0.283	1091	8728	13092
		6	0.477	4398	35184	52776
		7	1.086	18083	144664	216996
		8	2.001	81704	653632	980448
4	pumpkin.obj	4	0.907	110	8864	13296
		5	1.395	4603	36824	55236
		6	2.020	18608	148864	223296
		7	3.256	74624	596992	895488
		8	6.535	298959	2391672	3587508
5	pine.obj	4	0.527	525	4200	6300
		5	0.843	2725	21800	32700
		6	1.496	12491	99928	149892
		7	2.287	47069	376552	564828
		8	6.189	162573	1300584	1950876

Waktu eksekusi dipengaruhi oleh dua hal utama: jumlah *face* pada objek input dan kedalaman maksimal. Waktu eksekusi kira-kira berbanding lurus dengan jumlah *face* dan bertambah sekitar 2 kali lipat setiap pertambahan kedalaman maksimal. Walau mungkin kompleksitas waktu untuk kasus terburuk bisa O(8^k * *f*) dengan k = kedalaman maksimal dan *f* = jumlah *face*, kompleksitas waktu rata-rata mungkin mendekati Θ(2^k * *f*).

Kesimpulan dan Saran

Strategi algoritma *divide and conquer* adalah strategi yang cocok untuk melakukan *voxelization*. Optimasi dapat dilakukan pada kedalaman sebelumnya sebelum dihitung lagi pada kedalaman selanjutnya.

Adapun beberapa hal yang dapat dilakukan untuk mengoptimasi objek hasil perhitungan:

- Menggabungkan verteks dari dua kubus bersebelahan untuk mengurangi jumlah verteks yang ada.
- Menghilangkan *face/triangle* yang tidak terlihat supaya jumlah *face/triangle* berkurang.
- Membuat ulang *face/triangle* pada permukaan datar supaya jumlah *face/triangle* berkurang.
- Mencoba menambah jumlah thread yang ada supaya program lebih cepat lagi.

Renderer juga perlu dioptimasi lagi supaya kuat untuk me-*render* objek yang cukup besar.

Lampiran

Link repository github:

https://github.com/NafisKreatif/Tucil2_13524018_13524040

Tabel poin-poin yang diimplementasikan pada program:

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	V	
2	Program berhasil dijalankan	V	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	V	
4	Program informasi-informasi atau report	V	
5	Program dapat menerima masukan .obj, dan mengembalikan output	V	
6	[Bonus] Program menggunakan *concurrency*	V	
7	[Bonus] Program dapat menampilkan visualisasi .obj	V	

Pernyataan penggunaan AI

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Nafis Habibi

Kloce Paul William Saragih